

华中科技大学

编译原理实验报告

专 业： 计算机科学与技术
班 级： CS23
学 号： U2023xxxxx
姓 名： losyi
电 话：
邮 箱：

独创性声明

本人郑重声明本报告内容，是由作者本人独立完成的。有关观点、方法、数据和文献等的引用已在文中指出。除文中已注明引用的内容外，本报告不包含任何其他个人或集体已经公开发表的作品成果，不存在剽窃、抄袭行为。

特此声明!

作者签名:

日期: 2026 年 6 月 14 日

综合成绩	
教师签名	

目 录

1 编译工具链的使用	1
1.1 实验任务	1
1.2 实验实现	1
2 词法分析	7
2.1 实验任务	7
2.2 词法分析器的设计与实现	7
3 语法分析	10
3.1 实验任务	10
3.2 语法分析器的设计与实现	10
4 静态语义分析	15
4.1 实验任务	15
4.2 静态语义分析的设计与实现	15
5 中间代码生成	25
5.1 实验任务	25
5.2 中间代码生成器的实现	25
6 目标代码生成	29

华中科技大学实验报告

6.1 实验任务	29
6.2 目标代码生成器的设计与实现	29
7 总结	32
7.1 实验感想	32
7.2 实验总结与展望	32

1 编译工具链的使用

1.1 实验任务

- (1) 编译工具链的使用;
- (2) Sysy 语言及运行时库;
- (3) 目标平台 arm 的汇编语言;
- (4) 目标平台 riscv64 的汇编语言;

以上任务中(1)(2)为必做任务, (3)(4)中任选一个完成即可。

1.2 实验实现

1.编译工具链的使用

本关要求使用 gcc 编译器, 通过合适的命令行选项, 编译 def-test.c 和 alibaba.c, 生成可执行文件 def-test。程序运行时需要输出 Bilibili 的自我介绍以及 Alibaba 对 Bilibili 的喊话, 即源码中 #ifdef BILIBILI 控制的代码段必须被编译进去。

```
gcc def-test.c alibaba.c -o def-test -DBILIBILI
```

在 GCC 编译命令 gcc def-test.c alibaba.c -o def-test -DBILIBILI 中, -DBILIBILI 选项定义宏以保留条件编译代码 (如 Bilibili 自我介绍), 否则排除; -o def-test 指定输出文件名, 源文件自动包含头文件。

2.Clang 编译器的使用

本关给定一个 SysY2022 语言的源程序, 其中包含可优化的代码。要求使用 clang 编译器, 开启优化级别 O2, 将源程序翻译成 ARMv7 架构、Linux 系统、符合 gnuabihf 嵌入式 ABI 且支持硬浮点的汇编代码。

```
clang -S -O2 -target armv7-linux-gnueabi bar.c -o bar.clang.arm.s
```

使用 Clang 编译器命令编译, -S 表示生成汇编代码而不链接; -O2 启用二级优化 (包括常数折叠、死代码删除、函数简化等); -target armv7-linux-gnueabi 指定 ARMv7 架构及硬件浮点参数传递; 输入源文件为 bar.c, 输出汇编文件名为 bar.clang.arm.s。

3.交叉编译与 QEMU 模拟

本关任务要求将 `iplusf.c` 编译为 ARM 可执行文件,再用 `qemu-arm` 模拟运行。关键工具链包括 Linaro 提供的 `arm-linux-gnueabi-hf-gcc` 交叉编译器和 QEMU 的用户态模拟器。

```
# 1. 生成 ARM 汇编代码
arm-linux-gnueabi-hf-gcc -S -O2 -march=armv7 iplusf.c -o iplusf.arm.s
# 2. 汇编并链接运行时库,生成可执行文件(静态链接)
arm-linux-gnueabi-hf-gcc iplusf.arm.s -o iplusf.arm sylib.a -static
# 3. 用 QEMU 运行
qemu-arm -L /usr/arm-linux-gnueabi-hf/ iplusf.arm
```

实际操作分三步。第一步用交叉编译器生成 ARMv7 汇编代码, `-march=armv7` 保证指令集兼容。第二步将汇编文件与 SysV 运行时静态库 `sylib.a` 链接, `-static` 强制静态链接,使得可执行文件不依赖宿主机的动态链接器就能在 QEMU 中运行。第三步是模拟执行。

4.Makefile 构建项目

本关任务要求编写 Makefile,使得执行 `make helloworld` 能生成可执行文件 `helloworld`。需要正确处理头文件路径 `include`,并编译 C++源文件。

```
# 编译器
CXX := g++
# 编译选项
CXXFLAGS := -Wall -g -O2 -std=c++11
# 头文件路径
CPPFLAGS := -Iinclude
# 目标文件列表
objects := main.o helloworld.o
# 最终可执行文件
helloworld: $(objects)
    $(CXX) $^ -o $@
# 使用隐含规则编译 .o 文件
%.o: %.cc
    $(CXX) $(CXXFLAGS) $(CPPFLAGS) -c $< -o $@
```

在 Makefile 中,首先通过变量定义指定编译器和选项: `CXX := g++` 使用简单赋值指定 C++编译器, `CXXFLAGS := -Wall -g -O2 -std=c++11` 开启警告、调试信息、二级优化和 C++11 标准, `CPPFLAGS := -Iinclude` 添加头文件搜索路径, `objects := main.o helloworld.o` 列出中间目标文件。规则 `helloworld: $(objects)` 表示可执行文件依赖这两个目标文件,链接命令为 `$(CXX) $^ -o $@`; 模式规

则%.o: %.cc 则定义任何.o 文件依赖于同名.cc 文件, 编译命令为\$(CXX) \$(CXXFLAGS) \$(CPPFLAGS) -c \$< -o \$@ (\$<为第一个依赖即.cc 文件)。

执行 make helloworld 时, make 会检查目标文件是否需要重新编译(通过模式规则), 然后链接生成 helloworld 可执行文件。需要注意的是, 该 Makefile 未显式声明对头文件 helloworld.hh 的依赖, 因此修改头文件后不会自动触发重编译; 但本实验要求简单, 当前规则已满足基本功能, 运行程序应输出 pass!。

5.SysY 语言与运行时库

掌握了编译工具链的使用后, 本小节进一步要求熟悉后续实验的目标语言——SysY2022, 并利用其运行时库编写一个实际的算法程序。

SysY 是由 C 语言的一个子集扩展而来的教学语言, 也是编译系统设计赛的标准语言。每个 SysY 程序存储为一个.sy 文件, 有且仅有一个 main 函数, 可包含全局变量、常量和若干其他函数。语言支持 int 和 float (两种基本类型, 以及以它们为元素的多维数组。一个重要的约束是 SysY 没有提供原生的输入输出语句——I/O 完全依赖运行时库函数实现, 包括 getint (读取整数)、getfloat (读取浮点数)、putint (输出整数)、putfloat (输出浮点数)、putch (输出字符) 等。

题目要求用 SysY 实现“买卖股票的最佳时机”算法: 给定 N 个交易日股价序列, 选择某天买入、未来某天卖出, 求最大利润。算法本身并不复杂——遍历过程中记录历史最低价, 同时计算当前价与最低价的差值并更新最大值, 一次遍历即可得到结果。但用 SysY 实现时, 几个语言层面的限制需要特别注意。

其一, SysY 没有 for 语句、没有++/--自增运算符, 也没有+=等复合赋值运算符。所有循环必须用 while 实现, 计数变量递增必须写成 $i = i + 1$ 。其二, 输入操作由 getint()完成, 它不是 C 标准库的 scanf, 而是 SysY 运行时库提供的函数, 从标准输入读取一个整数并返回。同样, 输出整数用 putint(val), 输出单个字符用 putch(ascii_code), 输出换行需要显式调用 putch(10)。

我编写的程序定义了常量 N=10 和全局数组 prices[N], 在 main 中通过 while 循环调用 getint 读入股价, 然后调用 maxProfit 计算并输出结果。maxProfit 函数维护 minPrice (历史最低买价) 和 maxProfit_val (当前最大利润), 对每个价格

判断是否更新最低价或计算可能利润。

```
int maxProfit(int prices[]){
    int minPrice = prices[0];
    int maxProfit_val = 0;
    int i = 1;

    while (i < N) {
        if (prices[i] < minPrice) {
            minPrice = prices[i];
        } else {
            int profit = prices[i] - minPrice;
            if (profit > maxProfit_val) {
                maxProfit_val = profit;
            }
        }
        i = i + 1;
    }

    return maxProfit_val;
}
```

6.RISC-V 汇编基础

本章最后一项任务涉及 RISC-V 汇编语言，要求完善一个对整数数组进行冒泡排序的汇编程序。RISC-V 是近年来迅速崛起的开源指令集架构，也是本实验的目标代码生成的目标平台，提前熟悉其寄存器和指令体系对后续实验很有帮助。

RISC-V 的寄存器体系遵循精简设计原则，共 32 个通用寄存器，各有约定的用途。x0 恒为零，写入无效。ra(x1)存放函数返回地址，sp(x2)是栈指针。t0t6 为临时寄存器，由调用者负责保存；s0s11 为保存寄存器，被调用者使用前需备份并在返回前恢复。a0~a7 用于传递函数参数和返回值——前两个参数通过 a0 和 a1 传递，返回值也放在 a0 中。理解了这套约定，就能看懂函数调用时参数如何流动、寄存器值如何保护。

汇编指示用于指导汇编器的行为而非生成机器指令。.text 指示后续内容放入代码段，.globl 将符号声明为全局可见（类似于 C 中的 extern），.align 控制内存对齐。在 bubblesort 的框架中，函数接收 a0（数组首地址）和 a1（元素个数），要求返回 0。

冒泡排序的汇编实现需要双层循环。外循环控制轮数（最多 n-1 轮），内循环逐对比较相邻元素。核心的地址计算利用了 RISC-V 的缩放寻址：由于 int 占 4

字节，第 j 个元素的地址偏移量为 $j*4$ ，通过 `slli t4, t2, 2` 实现左移两位即乘以 4，再用 `add` 加到数组基址上。比较和交换则通过 `lw` 读取两个相邻元素、`ble` 判断顺序、`sw` 写回完成。整个函数使用了三个标签（`loop_i`、`loop_j`、`end`）和十个临时寄存器，指令虽简单，但寄存器分配和循环边界的计算需要仔细推敲——尤其是内循环上界为 $n-1-i$ ，每轮外循环后减少一次比较，这正是冒泡排序优化的关键。

下面是带注释的完整代码：

```
.text
.align 1
.globl bubblesort      # 声明 bubblesort 为全局符号，供 main 调用
.type bubblesort, @function

bubblesort:
    # 函数入口
    # 参数: a0 = 数组首地址, a1 = 数组元素个数 n
    # 返回值: a0 = 0

    # 外循环变量 i, 从 0 开始
    li      t0, 0        # t0 = i = 0
    addi    t1, a1, -1    # t1 = n-1, 外循环最多需要 n-1 轮

loop_i:
    # 外循环条件: 若 i >= n-1, 排序完成, 跳转到 end
    bge     t0, t1, end

    # 内循环变量 j, 从 0 开始
    li      t2, 0        # t2 = j = 0
    # 内循环上界 = n-1-i
    sub     t3, t1, t0    # t3 = (n-1) - i

loop_j:
    # 内循环条件: 若 j >= (n-1-i), 结束本次内循环
    bge     t2, t3, next_i

    # 计算 arr[j] 和 arr[j+1] 的地址
    # 每个 int 占 4 字节, 偏移 = j * 4
    slli    t4, t2, 2     # t4 = j * 4
    add     t5, a0, t4    # t5 = &arr[j] (arr[j] 的地址)
    lw      t6, 0(t5)     # t6 = arr[j]
    lw      a2, 4(t5)     # a2 = arr[j+1] (地址 +4)

    # 若 arr[j] <= arr[j+1], 顺序正确, 跳过交换
    ble     t6, a2, no_swap

    # ----- 交换 arr[j] 和 arr[j+1] -----
    sw      t6, 4(t5)     # arr[j+1] = arr[j]
    sw      a2, 0(t5)     # arr[j] = arr[j+1]

no_swap:
    # 内循环 j 递增
    addi    t2, t2, 1
    # 内循环条件: 若 j >= (n-1-i), 结束本次内循环
    bge     t2, t3, next_i

next_i:
    # 外循环 i 递增
    addi    t0, t0, 1
    # 外循环条件: 若 i >= n-1, 排序完成, 跳转到 end
    bge     t0, t1, end

end
```

华中科技大学实验报告

```
sw    a2, 0(t5)    # arr[j] = arr[j+1]
sw    t6, 4(t5)    # arr[j+1] = arr[j]

no_swap:
    addi    t2, t2, 1    # j++
    j       loop_j      # 继续内循环

next_i:
    addi    t0, t0, 1    # i++
    j       loop_i      # 继续外循环

end:
    li      a0, 0        # 返回值 0
    ret

.size    bubblesort, .-bubblesort
```

2 词法分析

2.1 实验任务

利用 flex 工具生成 SysY2022 语言的词法分析器，要求：

- (1) 输入一个 SysY2022 源程序文件（如 test.c）。
- (2) 输出该程序的 token 及种别，格式为：识别出的单词：种别名称（冒号前后各留一个空格）。
- (3) 对于词法错误，输出：Lexical error - line 行号：识别出来的串。
- (4) 直接忽略空白符（空格、换行、制表符）和注释。

2.2 词法分析器的设计与实现

flex 是一个经典的词法分析器生成工具。使用者在扩展名为.l 的规则文件中，用正则表达式描述每种 token 的模式，flex 会将其编译为确定性有限自动机（DFA），并输出一个包含 yylex() 函数的 C++ 源文件。每调用一次 yylex()，自动机就从输入流中识别出一个最长匹配的 token 并返回。

flex 源文件由 %% 分隔为三个区域。定义区用于声明 C++ 头文件、定义正则宏和设置选项（如开启行号追踪）。规则区是核心，由“模式{动作}”对组成。用户代码区放置 main 函数等辅助逻辑。

flex 遵循两个关键匹配原则。一是最长匹配原则：当多个模式都能匹配当前输入时，flex 会选择匹配字符数最多的那个。这是保证 1.2 被整体识别为浮点数（而非拆成 1、.、2 三个 token）的根本机制。二是优先匹配原则：当两个模式匹配长度相同时，写在文件前面的规则优先。因此关键字规则必须排在标识符规则之前——否则 int 会被误识别为普通 ID。

本试验要求在给定的基础代码上，补充完善以下规则及对应动作：

- (1) 标识符 ID 的识别规则。

- (2) int 型字面量 INT_LIT 的识别规则。
- (3) float 型字面量 FLOAT_LIT 的识别规则。
- (4) 词法错误 (如 9ab、2f、096) 的阻截与报错规则。

1.正则宏的设计

整数字面量区分三种进制。十进制 DEC_INT 定义为`[1-9][0-9]`，首字符禁止为 0 以避免与八进制混淆；八进制 OCT_INT 定义为`0[0-7]`；十六进制 HEX_INT 定义为`0[xX][0-9a-fA-F]+`，标志前缀 0x 后至少需要一个有效数字。

浮点数字面量是正则设计中最复杂的部分。我将其拆分为四个宏以覆盖所有合法形式。FLOAT_DEC_1 匹配带小数点的形式 (如 .04、2.0、3.f)，核心结构 `([0-9]*.[0-9]+|[0-9]+\.)` 用一个或操作同时覆盖了“.04”和“3.”两种写法。可选的指数部分 `([eE][+-]?[0-9]+)?` 和可选的 f/F 后缀为整条规则提供了灵活性。

FLOAT_DEC_2 处理不含小数点但带有指数的形式 (如 1e3、4e-04f)。十六进制浮点数 FLOAT_HEX_1 和 FLOAT_HEX_2 与之结构类似，但十六进制字符集为`[0-9a-fA-F]`，指数标志为 p/P 而非 e/E。

标识符 IDENT 定义为`[a-zA-Z_][a-zA-Z0-9_]*`，首字符限定为字母或下划线，后续可跟字母、数字或下划线，与 SysY2022 语言规范完全一致。

```
/* 1. 整数常量正则表达式 */
DEC_INT      [1-9][0-9]*
OCT_INT      0[0-7]*
HEX_INT      0[xX][0-9a-fA-F]+

/* 2. 浮点常量正则表达式 */
FLOAT_DEC_1 ([0-9]*\.[0-9]+|[0-9]+\.)([eE][+-]?[0-9]+)?[fF]?
FLOAT_DEC_2 [0-9]+[eE][+-]?[0-9]+[fF]?
FLOAT_HEX_1  0[xX]([0-9a-fA-F]*\.[0-9a-fA-F]+|[0-9a-fA-F]+\.)([pP][+-]?[0-9]+)[fF]?
FLOAT_HEX_2  0[xX][0-9a-fA-F]+[pP][+-]?[0-9]+[fF]?

/* 3. 标识符正则表达式 */
IDENT        [a-zA-Z_][a-zA-Z0-9_]*

/* 4. 注释匹配规则 */
MultilineComment  "/*"([^\*](\*+)[^\*/*])*(\*+)"
SingleLineComment "//"([^\n])*
```

2.规则顺序与错误阻截

规则区的排列顺序直接决定了识别效果。关键字规则 (int、float、if、while 等) 位于最前, 其后是标识符规则, 然后是整数字面量、浮点数字面量。必须将浮点数规则放在整数规则之前——例如输入 1.2, 浮点数规则能匹配三位字符 (1.2), 而整数规则只能匹配一位(1), 根据最长匹配原则 flex 会选择浮点数规则。

词法错误阻截使用一条精心构造的兜底规则: `[0-9][a-zA-Z0-9_\.][0-9][a-zA-Z0-9_\.]`。其逻辑是: 以数字开头 (或以小数点后跟数字开头) 但后面混入了字母等非法字符的串, 必然无法被上方任何合法数字规则匹配, 最终落入这里被报告为词法错误。合法的数字会在此之前被“截胡”。096 之所以报错, 是因为它在八进制规则 `OCT_INT` 中匹配到 0 后就停住了 (9 不在 0-7 范围内), 剩余 96 无法形成合法 token, 于是整体落入错误阻截规则。

最后一条规则是通配符., 作为终极兜底, 捕获任何上面所有规则都无法匹配的孤立非法字符。

```
/ ----- 1. 关键字与符号匹配 ----- /
"int" {printf("%s : INT\n", yytext); return INT; }
"float" {printf("%s : FLOAT\n", yytext); return FLOAT; }
"void" {printf("%s : VOID\n", yytext); return VOID; }
"const" {printf("%s : CONST\n", yytext); return CONST; }
"return" {printf("%s : RETURN\n", yytext); return RETURN; }
"if" {printf("%s : IF\n", yytext); return IF; }
"else" {printf("%s : ELSE\n", yytext); return ELSE; }
....此处省略若干关键字
/ ----- 2. 标识符匹配 ----- /
{IDENT} {printf("%s : ID\n", yytext); return ID;}
/ ----- 3. 数字字面量匹配 ----- /
{DEC_INT}{OCT_INT}{HEX_INT} {printf("%s : INT_LIT\n", yytext); return INT_LIT;}
{FLOAT_DEC_1}{FLOAT_DEC_2}{FLOAT_HEX_1}{FLOAT_HEX_2} {printf("%s :
FLOAT_LIT\n", yytext); return FLOAT_LIT;}
/ ----- 4. 词法错误阻截 ----- /
[0-9][a-zA-Z0-9_\.][0-9][a-zA-Z0-9_\.] {printf("Lexical error – line %d : %s\n",
yylineno, yytext); return LEX_ERR;}
/ ----- 5. 直接丢弃空白符及注释 ----- /
{MultilineComment} {}
{SingleLineComment} {}
[ \r\n\t]+ {}
/ ----- 6. 兜底错误 ----- /
. {printf("Lexical error – line %d : %s\n",yylineno,yytext);return LEX_ERR;}
```

3 语法分析

3.1 实验任务

使用 Flex 和 Bison 为 SysY2022 语言构造语法分析器。实验分为两个阶段：

第一阶段进行语法检查，对有语法错误的源程序识别错误并输出定位信息；

第二阶段对语法正确的程序构造抽象语法树（AST）并打印输出。实验框架中 tokens.l 描述词法规则，parser.y 描述语法规则和语义动作，ast.h 定义 AST 节点类体系。本次任务的核心是补全语句（Stmt）的 11 条产生式及其语义动作。

3.2 语法分析器的设计与实现

从编译器前端流程来看，本实验处在词法分析和语法分析阶段：

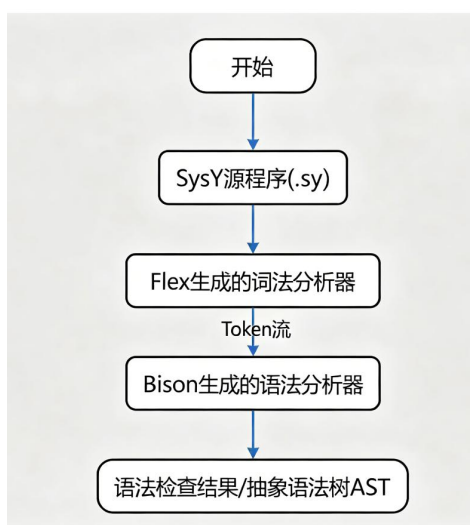


图 1 源程序编译流程图

在本实验框架中，tokens.l 描述词法规则，parser.y 描述语法规则和语义动作，ast.h 定义 AST 节点，printer.cpp 负责打印 AST。本次任务的核心是补全语句 Stmt 的产生式和语义动作

```
Stmt → LVal '=' Exp ';'
| [Exp] ';'
| Block
```

华中科技大学实验报告

```
| 'if' '(' Cond ')' Stmt [ 'else' Stmt ]  
| 'while' '(' Cond ')' Stmt  
| 'break' ';' ;  
| 'continue' ';' ;  
| 'return' [Exp] ';' ;
```

Bison 是语法分析器生成器。它读取上下文无关语法规则，生成 `yyparse()` 函数。`yyparse()` 调用 Flex 的 `yylex()` 获取 Token，并根据文法判断 Token 序列能否组成合法程序。

Bison 中一条语法规则通常写成：

```
非终结符:  
  符号 1 符号 2 ... 符号 n { 语义动作 }  
;
```

例如赋值语句：

```
Stmt:  
  LVal ASSIGN Exp SEMICOLON {  
    /* 构造赋值语句对应的 AST 节点 */  
  }  
;
```

如果目标只是通过语法错误检测，那么每条 Stmt 规则只要能被 Bison 识别即可。语义动作可以临时简化为创建空节点：

```
Stmt:  
  LVal ASSIGN Exp SEMICOLON { $$ = new StmtAST(); }  
| Exp SEMICOLON { $$ = new StmtAST(); }  
| SEMICOLON { $$ = new StmtAST(); }  
| Block { $$ = new StmtAST(); }  
| IF LP Cond RP Stmt %prec LOWER_THEN_ELSE { $$ = new StmtAST(); }  
| IF LP Cond RP Stmt ELSE Stmt { $$ = new StmtAST(); }  
| WHILE LP Cond RP Stmt { $$ = new StmtAST(); }  
| BREAK SEMICOLON { $$ = new StmtAST(); }  
| CONTINUE SEMICOLON { $$ = new StmtAST(); }  
| RETURN SEMICOLON { $$ = new StmtAST(); }  
| RETURN Exp SEMICOLON { $$ = new StmtAST(); }  
;
```

这样写可以完成“语法是否正确”的判断，但不能正确输出 AST。第二阶段必须补全每条语义动作。每一条产生式右侧的 {} 内是构造对应 AST 节点的语义动作：

(1) 赋值语句

赋值语句对应文法 `LVal '=' Exp ';'` ，我将左值 `$1` 和表达式 `$3` 分别存入 `StmtAST` 的 `lval` 和 `exp` 成员：

Stmt:

```
LVal ASSIGN Exp SEMICOLON {
    $$ = new StmtAST();           // 创建语句外壳节点
    $$->sType = ASS;              // 设置语句类型为赋值语句
    $$->lVal = unique_ptr<LValAST>($1); // 保存左值, 转移所有权
    $$->exp = unique_ptr<AddExpAST>($3); // 保存右侧表达式, 转移所有权
}
```

该规则的关键点在于:

ASS 枚举值告诉打印器这是一条赋值语句

使用 `unique_ptr` 接管子节点所有权, 避免内存泄漏

(2) 表达式语句与空语句

根据 SysY 文法的 `[Exp]'`, 表达式可缺省, 因此分为两条规则:

```
| Exp SEMICOLON {
    $$ = new StmtAST();
    $$->sType = EXP;           // 表达式语句类型
    $$->exp = unique_ptr<AddExpAST>($1); // 保存表达式
}
| SEMICOLON {
    $$ = new StmtAST();
    $$->sType = SEMI;         // 空语句类型, 无子节点
}
```

这里特别注意区分 `;` (空语句) 和 `{}` (空块): 前者设置 `sType = SEMI`, 后者由 `Block` 规则处理并设置 `sType = BLK`。

(3) 语句块

语句块本身已在 `Block` 规则中完成解析, 这里只需包装成 `StmtAST`:

c

```
| Block {
    $$ = new StmtAST();
    $$->sType = BLK;
    $$->block = unique_ptr<BlockAST>($1); // 直接包装已有的 BlockAST
}
```

(4) if 与 if-else 语句

这是实现中需要特别注意悬空 `else` 问题的部分:

```
| IF LP Cond RP Stmt %prec LOWER_THEN_ELSE {
    $$ = new StmtAST();
    $$->sType = SEL;
    SelectStmtAST* node = new SelectStmtAST();
    node->cond = unique_ptr<LORExpAST>($3); // 条件表达式
    node->ifStmt = unique_ptr<StmtAST>($5); // then 分支
    // elseStmt 保持 nullptr
    $$->selectStmt = unique_ptr<SelectStmtAST>(node);
}
| IF LP Cond RP Stmt ELSE Stmt {
```

```
$$ = new StmtAST();
$$->sType = SEL;
SelectStmtAST* node = new SelectStmtAST();
node->cond = unique_ptr<LOrExpAST>($3);
node->ifStmt = unique_ptr<StmtAST>($5);
node->elseStmt = unique_ptr<StmtAST>($7); // else 分支
$$->selectStmt = unique_ptr<SelectStmtAST>(node);
}
```

%prec LOWER_THEN_ELSE 配合优先级声明, 使 else 总是匹配最近的 if 无 else 时 elseStmt 自然为 nullptr, 打印器据此判断是 if 还是 if-else

(5) while 语句

while 节点需要保存两部分: cond (循环条件) 和 stmt(循环体), 循环体可以是单条语句, 也可以是语句块。语法阶段只负责识别 while (Cond) Stmt 形式。至于 break 和 continue 是否真的位于循环内部, 通常属于后续语义分析阶段处理。

```
c
| WHILE LP Cond RP Stmt {
    $$ = new StmtAST();
    $$->sType = ITER;
    IterationStmtAST* node = new IterationStmtAST();
    node->cond = unique_ptr<LOrExpAST>($3); // 循环条件
    node->stmt = unique_ptr<StmtAST>($5); // 循环体
    $$->iterationStmt = unique_ptr<IterationStmtAST>(node);
}
```

(6) break 与 continue

这两类语句无需子节点, 仅设置类型即可:

```
| BREAK SEMICOLON {
    $$ = new StmtAST();
    $$->sType = BRE;
}
| CONTINUE SEMICOLON {
    $$ = new StmtAST();
    $$->sType = CONT;
}
```

(7) return 语句

分为无返回值和有返回值两种情况:

return; 的 exp 为空, 通常用于 void 函数。return Exp; 的 exp 指向返回值表达式。函数返回类型与返回语句是否匹配, 属于语义分析内容, 不是本实验重点。

```
| RETURN SEMICOLON {
    $$ = new StmtAST();
    $$->sType = RET;
}
```

```
ReturnStmtAST* node = new ReturnStmtAST();
node->exp = nullptr; // 无返回值
$$->returnStmt = unique_ptr<ReturnStmtAST>(node);
}
| RETURN Exp SEMICOLON {
    $$ = new StmtAST();
    $$->sType = RET;
    ReturnStmtAST* node = new ReturnStmtAST();
    node->exp = unique_ptr<AddExpAST>($2); // 返回值表达式
    $$->returnStmt = unique_ptr<ReturnStmtAST>(node);
}
```

4 静态语义分析

4.1 实验任务

本实验要求在已构造好的 SysY2022 抽象语法树 (AST) 上实现静态语义检查器。语法分析只能判断程序结构是否合法，而静态语义检查需要进一步验证程序含义是否正确。本实验需要识别以下 10 种语义错误：

错误类型	触发场景
VarDuplicated	当前作用域重复定义变量，或函数形参重复定义
VarUnknown	使用未定义变量
FuncDuplicated	重复定义函数，或函数名与已有符号冲突
FuncUnknown	调用未定义函数，或把非函数当函数调用
FuncParamsNotMatch	函数调用的实参数量或类型与形参不匹配
FuncReturnTypeNotMatch	return 表达式类型与当前函数返回类型不匹配
ArrayIndexNotInt	数组下标表达式不是整型
BreakNotInLoop	break 不在循环内部
ContinueNotInLoop	continue 不在循环内部
VisitVariableError	对非数组变量使用下标访问，例如 a[0]

4.2 静态语义分析的设计与实现

实验给出的框架使用访问者模式 (Visitor Pattern)。它的核心思想是：AST 节点负责保存语法结构，Checker 负责执行语义检查。

每个 AST 节点一般都有一个 accept 方法：

```
cpp
void CompUnitAST::accept(Visitor &visitor) {
    visitor.visit(*this);
}
```

这行代码的含义是：节点把自己交给访问者，访问者根据实际节点类型调用对应的 visit 函数。

本实验中的 Checker 继承自 Visitor:

```
cpp
class Checker : public Visitor {
public:
    void visit(CompUnitAST &ast) override;
    void visit(FuncDefAST &ast) override;
    void visit(StmtAST &ast) override;
    void visit(LValAST &ast) override;
    void visit(CallAST &ast) override;
    // 其他 AST 节点的 visit 函数
};
```

这样设计有一个好处：以后如果要新增另一种 AST 操作，例如 AST 打印机、IR 生成器，不需要改 AST 类本身，只需要再写一个新的访问者即可。

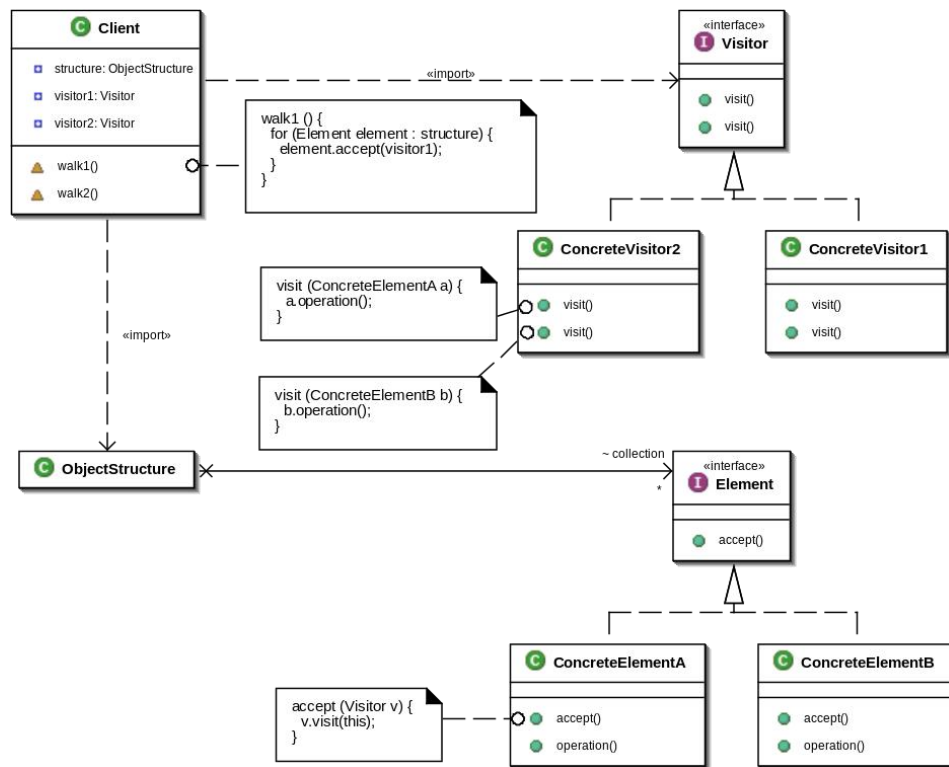


图 2 访问者模式架构图

除此之外，实验框架定义了一些符号表的基本操作，每个符号的详细信息由 Entry 保存:

```
struct Entry {
    bool is_array{};           // 是否是数组
    bool is_func{};           // 是否是函数
    TYPE type{};              // 变量类型，或函数返回类型
    int array_length{};        // 数组维数
    std::vector<int> arlen_value; // 每一维长度，函数形参数组第一维可用 -1 表示省略
    std::vector<Entry> func_params; // 函数形参列表
};
```

```
};
```

在 checker.h 文件中同样定义了关于符号表的几个方法。

`Entry *Lookup(const std::string& name)` 用于遍历并查询符号表是否含有某个符号（包含变量和函数）

`bool InsertVar(const DeclAST& node)` 在符号表中插入变量，注意这里的 node 节点里可能包含若干个变量定义（可以查看 DeclAST 类的定义），所以需要遍历 node 节点的 defList，插入时注意 Redefined Variable Error。

`bool InsertFunc(const FuncDefAST& node)` 在符号表中插入新的函数，插入新的函数需要在当前的 table 中加入新的符号表，要对函数的每一个参数遍历并插入到新的符号表中，同时也要注意检查是否有 Redefined Function Error，函数和形参均需插入到符号表中（注意是不同作用域，形参是在新创建的符号表中）。

`void make_new_table()` 插入新的表项，例如对于初始节点，或者进入某个新的作用域时需要插入新的表项。

`void delete_table()` 删除当前的表项。

下面具体说明如何检测某个错误。

1.变量重复定义

变量重复定义指的是在同一作用域内重复声明同名变量，或函数形参列表中出现重名参数。检查在向符号表插入时完成。

`InsertVar` 函数处理变量声明，遍历声明节点中的所有变量定义，逐一插入当前最内层作用域。先根据声明语句的基本类型和数组维度构建 `Entry` 结构体，如果变量是数组则遍历维度表达式并进行递归检查，确保维度是整型。构建完毕后调用 `table.front().insert()` 插入，若插入失败说明当前作用域已有同名符号，报告 `VarDuplicated` 错误。

```
bool InsertVar(const DeclAST &node) {
    for (auto &def : node.defList) {
        Entry tmp;
        tmp.is_func = false;
        tmp.type = node.bType;

        if (!def->arrays.empty()) {
            tmp.is_array = true;
            tmp.array_length = def->arrays.size();
        }
    }
}
```

```
        for (auto &exp : def->arrays) {
            if (exp) {
                exp->accept(*this);
                if (!Expr_int) {
                    report_and_exit(ErrorType::ArrayIndexNotInt, *def->id);
                }
            }
        }
    } else {
        tmp.is_array = false;
    }

    auto result = table.front().insert({*def->id, tmp});
    if (!result.second) {
        report_and_exit(ErrorType::VarDuplicated, *def->id);
    }
}
return true;
}
```

形参重复定义在 InsertFunc 中检查。函数形参被插入到为函数体新创建的作用域 func_table 中，若插入失败则报告同样的 VarDuplicated 错误。

```
cpp
auto param_result = func_table.insert({*param->id, param_entry});
if (!param_result.second) {
    report_and_exit(ErrorType::VarDuplicated, *param->id);
}
```

2.使用未定义变量与非数组下标访问

这两种错误在 visit(LValAST) 中集中检查。访问左值节点时先遍历下标表达式并递归检查，每访问完一个下标就检查 Expr_int 是否为真，下标不是整型则报告 ArrayIndexNotInt。下标检查完毕后调用 Lookup 查找变量名，找不到或者查到的是函数则报告 VarUnknown。最后判断非数组变量是否被错误地使用了数组下标访问语法，若是则报告 VisitVariableError。检查通过后将变量的类型写入 current_type，并同步更新 Expr_int。

```
void Checker::visit(LValAST &ast) {
    for (auto &exp : ast.arrays) {
        if (exp) {
            exp->accept(*this);
            if (!Expr_int) {
                report_and_exit(ErrorType::ArrayIndexNotInt, *ast.id);
            }
        }
    }
}
```

```
}
Entry *entry = Lookup(*ast.id);
if (entry == nullptr || entry->is_func) {
    report_and_exit(ErrorType::VarUnknown, *ast.id);
}
if (!entry->is_array && !last.arrays.empty()) {
    report_and_exit(ErrorType::VisitVariableError, *ast.id);
}
current_type = *entry;
Expr_int = (entry->type == TYPE::TYPE_INT);
}
```

3.函数重复定义

函数重复定义在 InsertFunc 中检查。构建好函数的 Entry 后将其插入外层作用域，若插入失败说明全局已有同名函数或变量，报告 FuncDuplicated 错误。

```
auto func_result = table.front().insert({*node.id, func_entry});
if (!func_result.second) {
    report_and_exit(ErrorType::FuncDuplicated, *node.id);
}
```

4.函数未定义或非函数调用

函数调用的检查集中在 visit(CallAST) 中。先处理运行时库函数，如 getint、putint 等，通过函数名硬编码识别，直接设置返回类型后返回。对于用户定义函数，调用 Lookup 查找函数名，若查找结果为空或找到的符号不是函数，则报告 FuncUnknown 错误。

```
void Checker::visit(CallAST &ast) {
    if (*ast.id == "getint" || *ast.id == "getch") {
        current_type.type = TYPE::TYPE_INT;
        Expr_int = true;
        return;
    }
    if (*ast.id == "putint" || *ast.id == "putch") {
        current_type.type = TYPE::TYPE_VOID;
        return;
    }

    Entry *entry = Lookup(*ast.id);
    if (entry == nullptr || !entry->is_func) {
        report_and_exit(ErrorType::FuncUnknown, *ast.id);
    }
    // 后续参数匹配检查...
}
```

5.函数参数不匹配

参数匹配检查紧接在函数存在性检查之后。先比较形参列表长度与实参列表长度是否一致，不一致则报告 `FuncParamsNotMatch`。数量一致时逐个访问实参表达式以更新 `current_type`，然后与对应形参比较基本类型和数组属性，任一不匹配则报告同样错误。检查通过后，将 `current_type` 更新为函数的返回类型。

```
if (entry->func_params.size() != ast.funcCParamList.size()) {
    report_and_exit(ErrorType::FuncParamsNotMatch, *ast.id);
}

for (size_t i = 0; i < ast.funcCParamList.size(); ++i) {
    ast.funcCParamList[i]->accept(*this);
    const Entry &formal = entry->func_params[i];
    if (current_type.type != formal.type ||
        current_type.is_array != formal.is_array) {
        report_and_exit(ErrorType::FuncParamsNotMatch, *ast.id);
    }
}

current_type = *entry;
Expr_int = (entry->type == TYPE::TYPE_INT);
}
```

6.返回值类型不匹配

在 `visit(FuncDefAST)` 中先将当前函数的返回类型记录到 `current_func_type`，然后进入函数体检查。在 `visit(ReturnStmtAST)` 中，先将返回类型默认设为 `TYPE_VOID`，若有返回表达式则访问表达式并获取其类型，最后比较实际返回类型与函数声明的返回类型是否一致，不一致则报告 `FuncReturnTypeNotMatch`。

```
void Checker::visit(FuncDefAST &ast) {
    current_func_type = ast.funcType;
    InsertFunc(ast);
    start_of_new_func = true;
    ast.block->accept(*this);
}

void Checker::visit(ReturnStmtAST &ast) {
    TYPE return_type = TYPE::TYPE_VOID;
    if (ast.exp) {
        ast.exp->accept(*this);
        return_type = current_type.type;
    }
    if (return_type != current_func_type) {
        report_and_exit(ErrorType::FuncReturnTypeNotMatch, "return");
    }
}
```

```
}
```

7.数组下标非整型

数组下标非整型的检查分布在变量声明的维度表达式和左值节点的下标访问中。两处的检查方式一致：访问完下标表达式后检查 `Expr_int` 标志，若为假则报告 `ArrayIndexNotInt` 错误。在 `LValAST` 中的相关代码已在 2 小节给出，在 `InsertVar` 中的检查代码位于 1 小节。`Expr_int` 由各个叶子表达式节点负责更新，整数字面量设为真，浮点数字面量设为假，变量左值和函数调用则根据其类型设置。

8.break 和 continue 不在循环内

该检查依赖继承属性 `is_inloop` 沿 AST 逐层传递。在 `visit(BlockAST)` 中，将当前块的 `is_inloop` 传递给每个子节点。

```
void Checker::visit(BlockAST &ast) {
    if (start_of_new_func) {
        start_of_new_func = false;
    } else {
        make_new_table();
    }
    for (auto &item : ast.blockItemList) {
        item->is_inloop = ast.is_inloop;
        item->accept(*this);
    }
    delete_table();
}
```

在 `visit(BlockItemAST)` 中，若当前元素是语句则继续向下传递 `is_inloop`。

```
void Checker::visit(BlockItemAST &ast) {
    if (ast.stmt) {
        ast.stmt->is_inloop = ast.is_inloop;
        ast.stmt->accept(*this);
    }
    if (ast.decl) {
        ast.decl->accept(*this);
    }
}
```

进入 `while` 循环体时，在 `visit(IterationStmtAST)` 中将循环体语句的 `is_inloop` 强制设为 `true`。

```
void Checker::visit(IterationStmtAST &ast) {
    if (ast.cond) ast.cond->accept(*this);
    if (ast.stmt) {
```

```
    ast.stmt->is_inloop = true;
    ast.stmt->accept(*this);
}
}
```

if 语句只继承父节点状态，不做修改。

```
void Checker::visit(SelectStmtAST &ast) {
    if (ast.cond) ast.cond->accept(*this);
    if (ast.ifStmt) {
        ast.ifStmt->is_inloop = ast.is_inloop;
        ast.ifStmt->accept(*this);
    }
    if (ast.elseStmt) {
        ast.elseStmt->is_inloop = ast.is_inloop;
        ast.elseStmt->accept(*this);
    }
}
```

最终在 visit(StmtAST) 中，根据 sType 判断当前是否为 break 或 continue，若 is_inloop 为假则报告对应错误。

```
void Checker::visit(StmtAST &ast) {
    if (ast.selectStmt) {
        ast.selectStmt->is_inloop = ast.is_inloop;
        ast.selectStmt->accept(*this);
    }
    if (ast.block) {
        ast.block->is_inloop = ast.is_inloop;
        ast.block->accept(*this);
    }
    if (ast.iterationStmt) {
        ast.iterationStmt->is_inloop = ast.is_inloop;
        ast.iterationStmt->accept(*this);
    }
    if (ast.returnStmt) ast.returnStmt->accept(*this);
    if (ast.lVal) ast.lVal->accept(*this);
    if (ast.exp) ast.exp->accept(*this);

    if (ast.sType == STYPE::BRE && !ast.is_inloop) {
        report_and_exit(ErrorType::BreakNotInLoop, "break");
    }
    if (ast.sType == STYPE::CONT && !ast.is_inloop) {
        report_and_exit(ErrorType::ContinueNotInLoop, "continue");
    }
}
```

9.表达式类型状态传递

表达式类型信息通过全局变量 current_type 和 Expr_int 在节点间传递。叶

子节点负责设置类型，中间节点负责递归向下访问。

数字字面量节点根据 `isInt` 标志设置类型。

```
void Checker::visit(NumberAST &ast) {
    Expr_int = ast.isInt;
    current_type.type = ast.isInt ? TYPE::TYPE_INT : TYPE::TYPE_FLOAT;
    current_type.is_array = false;
    current_type.is_func = false;
}
```

左值节点在完成查找后将符号类型写入 `current_type`。函数调用节点在完成匹配后将函数返回类型写入 `current_type`。二元表达式节点则递归访问左右子表达式。

```
cpp
void Checker::visit(AddExpAST &ast) {
    if (ast.addExp) ast.addExp->accept(*this);
    if (ast.mulExp) ast.mulExp->accept(*this);
}

void Checker::visit(MulExpAST &ast) {
    if (ast.mulExp) ast.mulExp->accept(*this);
    if (ast.unaryExp) ast.unaryExp->accept(*this);
}

void Checker::visit(UnaryExpAST &ast) {
    if (ast.primaryExp) ast.primaryExp->accept(*this);
    if (ast.unaryExp) ast.unaryExp->accept(*this);
    if (ast.call) ast.call->accept(*this);
}

void Checker::visit(PrimaryExpAST &ast) {
    if (ast.exp) ast.exp->accept(*this);
    if (ast.lval) ast.lval->accept(*this);
    if (ast.number) ast.number->accept(*this);
}
```

关系表达式和相等表达式在访问完子表达式后，将结果类型设为布尔型。

```
cpp
void Checker::visit(RelExpAST &ast) {
    if (ast.relExp) ast.relExp->accept(*this);
    if (ast.addExp) ast.addExp->accept(*this);
    Expr_int = false;
    current_type.type = TYPE::TYPE_BOOL;
}

void Checker::visit(EqExpAST &ast) {
    if (ast.eqExp) ast.eqExp->accept(*this);
}
```

```
if (ast.relExp) ast.relExp->accept(*this);  
Expr_int = false;  
current_type.type = TYPE::TYPE_BOOL;  
}
```

在实现过程中我遇到了三个典型问题。

第一个问题是函数体内找不到形参。原因在于 `visit(BlockAST)` 中无条件调用 `make_new_table()` 创建了新的空作用域，而 `InsertFunc` 已经为函数体创建了包含形参的作用域。新增的作用域隔在中间，导致形参无法被访问。解决方案是引入 `start_of_new_func` 标志，在 `InsertFunc` 完成作用域创建后将其置为 `true`，`visit(BlockAST)` 检测到该标志时跳过建表步骤。

第二个问题是 `while` 内部嵌套的 `if` 语句中的 `break` 被误报为不在循环内。原因在于 `is_inloop` 状态传递断链，`StmtAST` 没有将该状态传递给内部的 `selectStmt`。在 `visit(StmtAST)` 中增加对 `ast.selectStmt->is_inloop` 的显式赋值后问题解决。

第三个问题是数组下标表达式中的未定义变量被漏检。最初在 `visit(LValAST)` 中先查找变量再检查下标，若变量查找通过但下标中有未定义变量，该错误会被跳过。调整为先遍历下标表达式并递归检查，再查找变量本身，确保嵌套错误优先暴露。

5 中间代码生成

5.1 实验任务

本实验要求在给出的中间代码生成器框架基础上, 完成 LLVM IR 中间代码的生成, 将 SysY 语言程序翻译成 LLVM IR。具体任务是补全赋值语句的翻译逻辑, 使得编译器遇到 `lVal = exp;` 形式的赋值语句时, 能够生成正确的 `store` 指令, 并在左右类型不一致时自动插入类型转换指令。

5.2 中间代码生成器的实现

中间代码生成器采用访问者模式, 在 AST 上递归遍历并为每个节点生成对应的 LLVM IR。整个流程从 `main.cpp` 开始: 先调用 `yyparse()` 完成语法分析得到 AST, 然后创建 `GenIR` 访问者对象, 从根节点开始遍历整棵 AST。遍历过程中, 各类节点根据自身语义调用 IR 指令构造器生成对应的 `Instruction` 对象, 最终所有指令被组织在一个 `Module` 中, 打印输出即为文本形式的 LLVM IR。

```
main.cpp
yyin = fopen(filename, "r");
yyparse();
GenIR genIR;
root->accept(genIR);
std::unique_ptr m = genIR.getModule();
std::string IR = m->print();
if (print_ir) {
    std::cout << IR << std::endl;
}
```

本实验的核心代码位于 `genIR.cpp` 的 `visit(StmtAST &ast)` 函数中, 需要补全的是 `case ASS` 分支, 即处理赋值语句的部分。

实现赋值语句之前, 需要理解几个关键的全局状态变量。`requireLVal` 是一个布尔开关, 控制 `visit(LValAST)` 的返回内容——当它为真时, 左值节点返回变量的地址; 当它为假时, 左值节点生成 `load` 指令并返回变量中保存的值。`recentVal` 是一个全局的 `Value*` 指针, 保存最近一次访问任意节点后得到的结果, 相当于各 `visit` 函数之间传递结果的临时容器。`builder` 是指令构造器, 封装

华中科技大学实验报告

了 `create_store`、`create_load`、`create_fptosi`、`create_sitofp` 等常用指令的创建方法，自动将生成的指令插入当前基本块。

```
bool requireLVal = false; // 控制 LVal 节点返回地址还是返回值
Value recentVal = nullptr; // 保存最近一次 visit 得到的结果
IRStmtBuilder builder; // 用于创建 IR 指令
```

这三个状态的配合方式是：访问 AST 子节点前设置好 `requireLVal`，子节点的 `visit` 函数根据这个开关决定行为，执行完毕后将结果写入 `recentVal`，调用方再从中取出结果，必要时通过 `builder` 创建类型转换或 `store` 指令。

在 `ast.h` 中，语句节点定义如下：

```
class StmtAST : public BaseAST {
public:
    STYPE sType;
    unique_ptr<LValAST> lVal = nullptr;
    unique_ptr<AddExpAST> exp = nullptr;
    unique_ptr<ReturnStmtAST> returnStmt = nullptr;
    unique_ptr<SelectStmtAST> selectStmt = nullptr;
    unique_ptr<IterationStmtAST> iterationStmt = nullptr;
    unique_ptr<BlockAST> block = nullptr;
    void accept(Visitor &visitor) override;
};
```

`sType` 表示当前语句是哪一种语句。枚举定义在 `define.h`：

```
enum STYPE {
    SEMI, ASS, EXP, CONT, BRE, RET, BLK, SEL, ITER
};
```

其中 `ASS` 就表示赋值语句。

在 `visit(StmtAST &ast)` 中，只要遇到 `case ASS`，就应该：

- (1) 访问 `ast.lVal`，拿到左值地址；
- (2) 访问 `ast.exp`，拿到右值结果；
- (3) 检查左右类型是否匹配；
- (4) 生成 `store` 指令。

在 `case ASS` 中补充如下代码：

```
case ASS: {
    // ***** 代码填写处
    requireLVal = true;
    ast.lVal->accept(*this);
    auto addr = recentVal;

    ast.exp->accept(*this);
```

```
auto val = recentVal;
auto addrType = static_cast<PointerType*>(addr->type_)->contained_;
if (addrType == INT32_T && val->type_ == FLOAT_T) {
    val = builder->create_fptosi(val, INT32_T);
} else if (addrType == FLOAT_T && val->type_ == INT32_T) {
    val = builder->create_sitofp(val, FLOAT_T);
}
builder->create_store(val, addr);
// ***** 代码结束
break;
}
```

第一步是获取左值地址。在访问 `ast.lVal` 之前，将 `requireLVal` 设为 `true`，这样 `visit(LValAST)` 在查找符号表找到变量对应的 `alloca` 指令结果后，不会生成 `load`，而是直接把 `alloca` 产生的地址作为结果放入 `recentVal`。访问完成后，从 `recentVal` 中取出地址保存到局部变量 `addr` 中。这里的 `addr` 是一个指针类型，对于 `int a` 声明的变量，其类型是 `i32*`；对于 `float f`，则是 `float*`。`visit(LValAST)` 内部会自动将 `requireLVal` 重置为 `false`，所以不需要手动再改一次。

第二步是计算右值表达式。直接调用 `ast.exp->accept(*this)` 访问右侧表达式节点。表达式内部的各个子节点会递归生成对应的 `IR` 指令，最终的计算结果同样放入 `recentVal`。访问结束后将其保存到局部变量 `val` 中。右值可能是整数常量、浮点常量、某个变量的 `load` 结果、一次算术运算的结果或者函数调用的返回值。

第三步是取出赋值目标的实际类型。`addr` 的类型是指针，但 `store` 指令关心的是指针指向的类型。通过 `static_cast<PointerType*>(addr->type_)->contained_` 可以从指针类型中提取出被指向的类型。例如 `addr` 的类型是 `i32*`，提取出的就是 `i32`；如果类型是 `float*`，提取出的就是 `float`。这个目标类型用来与右值类型做比较。

第四步是处理类型不匹配的情况。`SysY` 允许 `int` 和 `float` 之间互相赋值，但 `LLVM IR` 的 `store` 指令要求右值类型与目标指针指向的类型严格一致。当目标类型是 `i32` 而右值是 `float` 时，需要插入 `fptosi` 指令将浮点数转换为整数；当目标类型是 `float` 而右值是 `i32` 时，需要插入 `sitofp` 指令将整数转换为浮点

数。如果左右类型本来就一致，则跳过转换步骤。类型转换指令通过 `builder` 创建，转换结果重新赋值给 `val`，替换原来的右值。

第五步是生成 `store` 指令。调用 `builder->create_store(val, addr)`，将右值写入左值地址。参数顺序是先值后地址，不能写反。`store` 指令内部有断言检查值的类型是否与指针指向的类型一致，如果前面没有正确做类型转换，这里就会触发断言失败。

6 目标代码生成

6.1 实验任务

本实验要求使用 LLVM 后端代码生成接口，将上一实验生成的 LLVM IR 文件翻译成 RISC-V 64 目标平台的汇编文件。核心任务是在给出的 `codegen.cc` 框架中补全三处关键代码：初始化 LLVM 目标注册表、指定目标平台为 `riscv64-unknown-elf`、准备输出文件流和 Pass 管理器。补全后，编译器能够读取 `.ll` 文件，自动生成对应的 `.s` 汇编文件。

6.2 目标代码生成器的设计与实现

实验已经给出了代码框架和提示信息（注释），目标代码生成器的工作流程十分清晰。首先通过 `parseIRFile` 读取上一实验生成的 LLVM IR 文件，将其解析为内存中的 `Module` 对象。然后初始化 LLVM 的所有目标后端，使 RISC-V 后端可用。接着设置目标三元组为 `riscv64-unknown-elf`，并通过 `TargetRegistry::lookupTarget` 查找对应的目标后端。找到后端后创建 `TargetMachine`，描述目标机器的 CPU、特性等配置。最后创建 Pass 管理器和输出文件流，调用 `addPassesToEmitFile` 添加代码生成 Pass，运行 Pass 将 IR 翻译为目标平台汇编并写入文件。

整个流程可以概括为：读取 IR 构造 `Module` → 初始化目标后端 → 指定目标平台 → 查找并创建目标机器 → 运行代码生成 Pass 输出汇编。

框架中的 `codeGenerate` 函数已经做好了 IR 解析（`parseIRFile` 读入 `.ll` 构造 `Module`）和错误处理，我需要补全三处空缺。下面按实际补全顺序逐一说明。

LLVM 支持多平台代码生成，但每个目标后端需要先注册才能使用。在 `codegen.cc` 中，`parseIRFile` 成功之后、`TargetRegistry::lookupTarget` 之前，需要插入以下五条初始化调用：

```
InitializeAllTargetInfos();
InitializeAllTargets();
```

```
InitializeAllTargetMCs();
InitializeAllAsmParsers();
InitializeAllAsmPrinters();
```

这五条调用各司其职。`InitializeAllTargetInfos` 注册各目标平台的基本描述信息（如 RISC-V 有 32 位和 64 位变体、支持哪些扩展）。`InitializeAllTargets` 注册目标后端本身，这是 `lookupTarget` 能够找到 `riscv64` 后端的必要条件。`InitializeAllTargetMCs` 注册机器码层的组件——后续输出汇编时 LLVM 需要知道 RISC-V 指令的编码格式。`InitializeAllAsmParsers` 和 `InitializeAllAsmPrinters` 分别注册汇编解析器和输出器，后者是本实验生成.s 文件的关键。如果遗漏这五行中的任何一行，后续 `lookupTarget` 都会因为注册表中找不到 `riscv64-unknown-elf` 而返回空指针，`codeGenerate` 将以失败返回。

目标三元组是 LLVM 用来描述目标平台的字符串，格式为 <架构>-<厂商>-<系统>-<ABI>。本实验评测使用 RISC-V 64 位平台，因此需要明确指定这个字符串。

```
// 补充代码 2: 指定目标平台
auto target_triple = "riscv64-unknown-elf";
```

`riscv64` 表示 64 位 RISC-V 架构，`unknown` 表示厂商未指定，`elf` 表示使用 ELF 可执行文件格式。框架中紧随其后有 `module->setTargetTriple(target_triple)`，这行代码将目标三元组写入 Module，后续的代码生成 Pass 会读取这个信息来决定生成哪种架构的汇编。

这里有一个容易出错的地方。如果写成 `auto target_triple = getDefaultTargetTriple()`，LLVM 会返回当前主机平台的三元组。在 x86 Linux 上会生成 x86 汇编，在 macOS 上会生成 ARM 或 x86 汇编，都不是评测要求的 RISC-V 汇编。因此必须硬编码为 `riscv64-unknown-elf`，与评测环境保持一致。

在框架中 `TheTargetMachine->addPassesToEmitFile(...)` 被调用之前，需要准备好它所需的三个参数——输出文件流 `dest`、Pass 管理器 `pass` 和输出文件类型 `file_type`：

```
std::string filename = getGenFilename(ir_filename, gen_filetype);
std::error_code EC;
```

```
raw_fd_ostream dest(filename, EC, sys::fs::OF_None);
if (EC) {
    errs() << "Could not open file: " << EC.message();
    return 1;
}
```

```
legacy::PassManager pass;
auto file_type = gen_filetype;
```

getGenFilename 是 codegen.h 中声明的辅助函数，它根据输入 IR 文件名和输出类型生成目标文件名——例如输入 012_func_defn.ll、输出类型 CGFT_AssemblyFile 时，返回 012_func_defn.s。raw_fd_ostream 是 LLVM 提供的文件输出流包装，EC 用于接收打开文件时的错误码。legacy::PassManager 是旧版 Pass 管理器，虽然 LLVM 已推出新版 PassManager，但 addPassesToEmitFile 目前仍使用旧版接口。

file_type 直接从函数参数 gen_filetype 取得——main.cc 调用 codeGenerate 时传入的就是 CGFT_AssemblyFile，表示生成汇编文件。如果传入 CGFT_ObjectFile 则会生成目标文件(.o)。

这三处代码补全后，框架中的 addPassesToEmitFile(pass, dest, nullptr, file_type) 会将代码生成 Pass 链添加到 PassManager，随后的 pass.run(*module)会依次运行指令选择、寄存器分配、指令调度、汇编输出等 Pass，最终将生成的 RISC-V 汇编代码写入 dest 指向的.s 文件。

7 总结

7.1 实验感想

本次编译原理实验让我完整地经历了一个编译器从前端到后端的全部流程，从词法分析、语法分析、静态语义分析，到中间代码生成和目标代码生成，每一步都让我对编译原理的理论知识有了更深刻的理解。

相比理论课的学习，实验让我们接触了更多新工具，新的设计模式和编码思想，尽管各个实验程序比较复杂，不过实验内容大多只聚焦在一小块，注释 (TODO) 部分讲解清晰，完成难度并不大。

实验 1 让我对编译工具链有了更系统的认识。此前我只知道 gcc 是一种编辑器，只知道 MakeFile 可以帮助我们快速构建程序。但具体命令怎么用，文件怎么编写，什么是交叉汇编，这些都是在实验的动手操作中才真正理解的。

实验 2 和实验 3 最为复杂。flex 的正则表达式和 bison 的产生式规则容易让人眼花缭乱。正则式和规则匹配需要我反复研究和纠错。

实验 4 让我认识了一种新的设计模式：访问者模式。本关实验代码量相对较大，需要我们对符号表，类型状态，语法树等在概念和代码实现上有清晰的认识。

实验 5 和 6 较为简单，主要是为了加深我们对后端的理解，不再赘述。

我看到实验课堂后面还加入了 Rust-Kunpeng 编译器实践，更具前瞻性和挑战性，体现了课程设计者对技术趋势的敏锐把握对学生工程能力的更高期待。不过，实验文档中涉及 Rust 特性但缺少解释，缺乏 Kunpeng/AArch64 平台背景知识和应用，建议后续持续完善文档和调试支持，使之成为可推广的编译原理实验标杆。

7.2 实验总结与展望

总之，这次实验我走完了编译器从前端到后端的完整流程，把课本里抽象的编译知识落到了实操上。我了解了各类编译工具、词法语法分析、语义

检查以及代码生成的基本实现思路，也摸清了编译器每一个阶段的工作逻辑。实操过程中我也发现了不少短板，比如调试文法、正则表达式时效率不高，对符号表、AST 状态传递的细节理解还不够扎实，对汇编和 LLVM 底层优化也只是浅尝辄止。

接下来我会继续夯实编译原理基础，吃透核心知识点。同时在现有代码上尝试添加简单优化功能，深入学习 RISC-V 指令集与 LLVM 相关技术，多动手实践，进一步提升自己的底层开发能力。